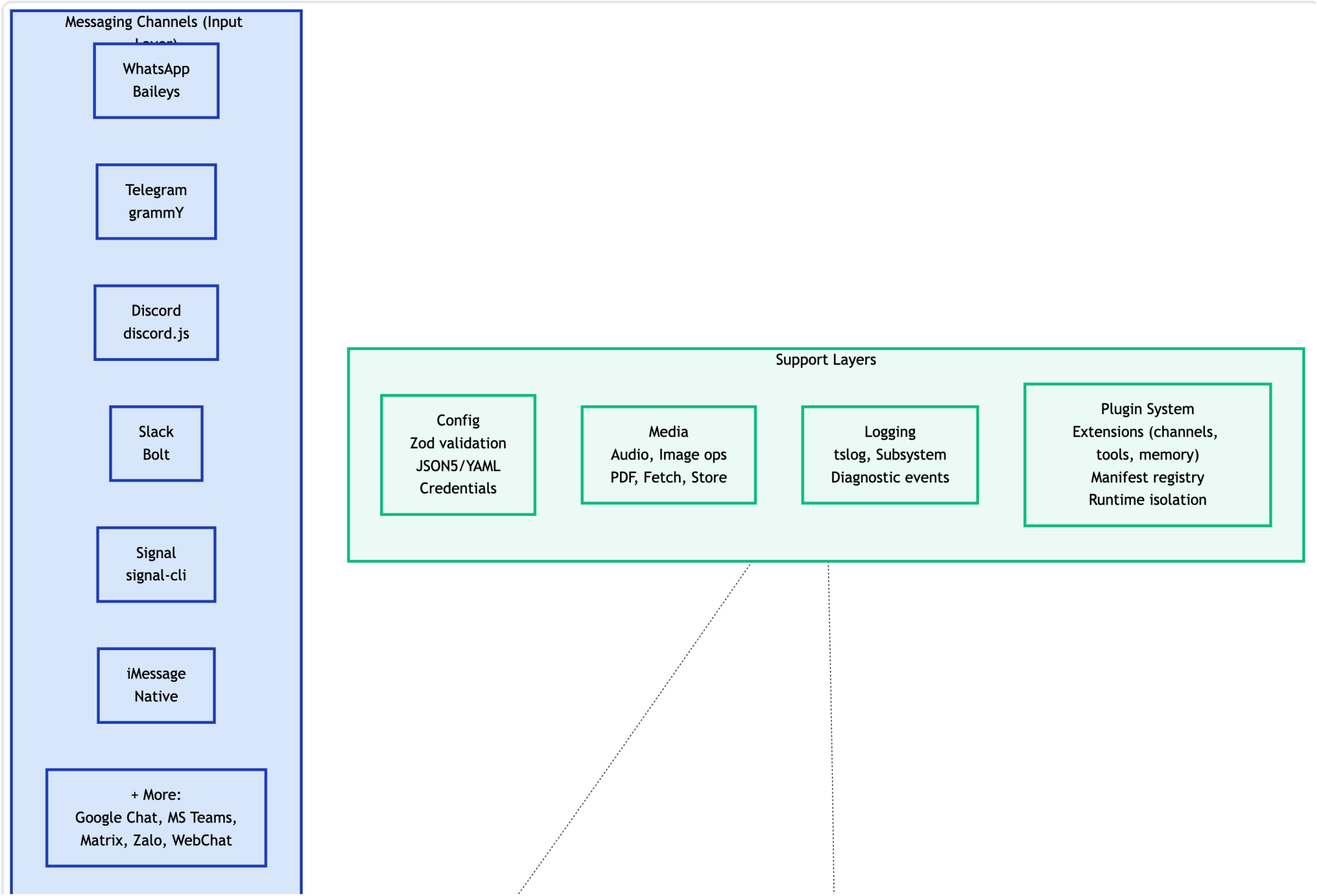
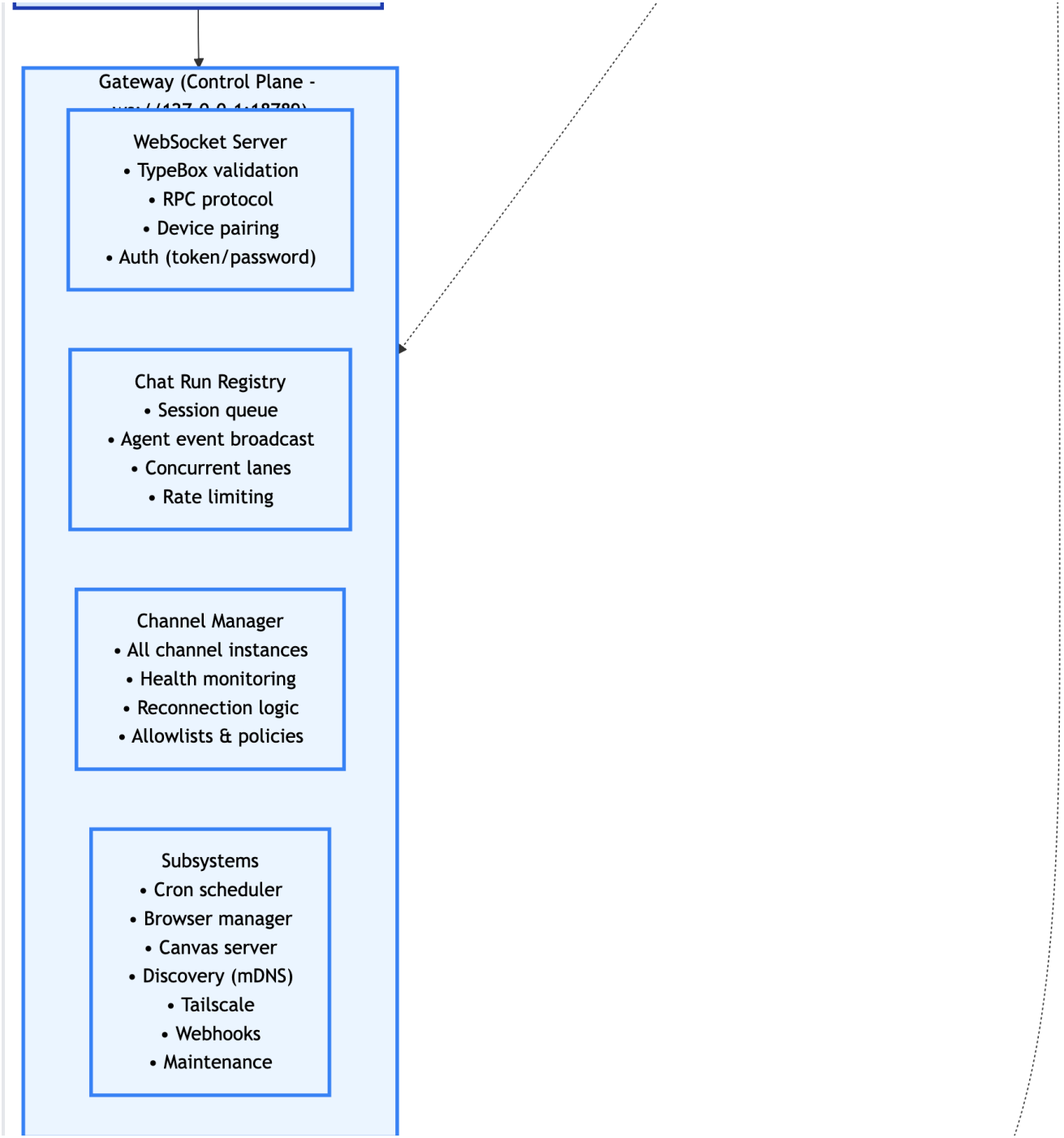
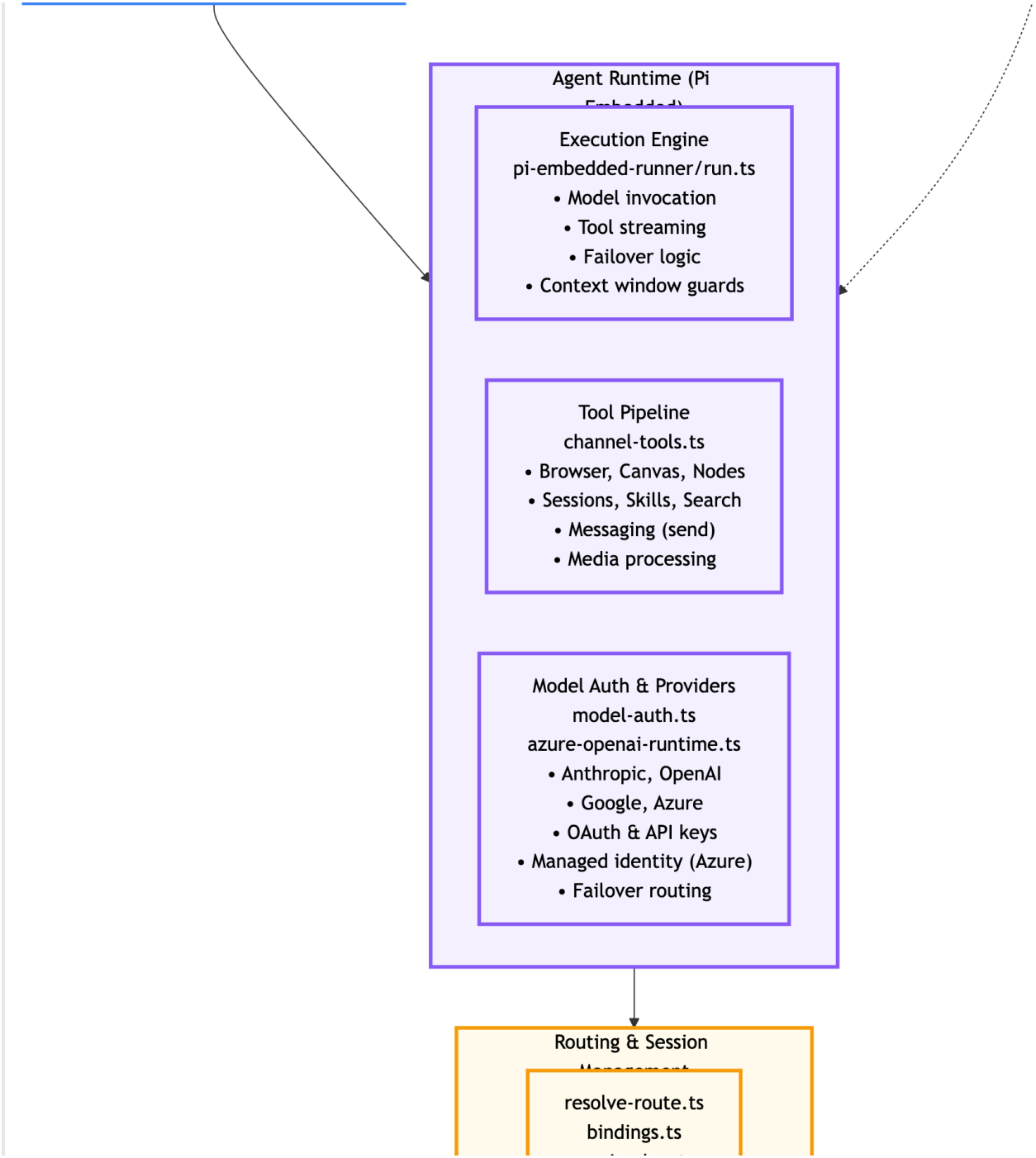
Clawdbot - System Architecture

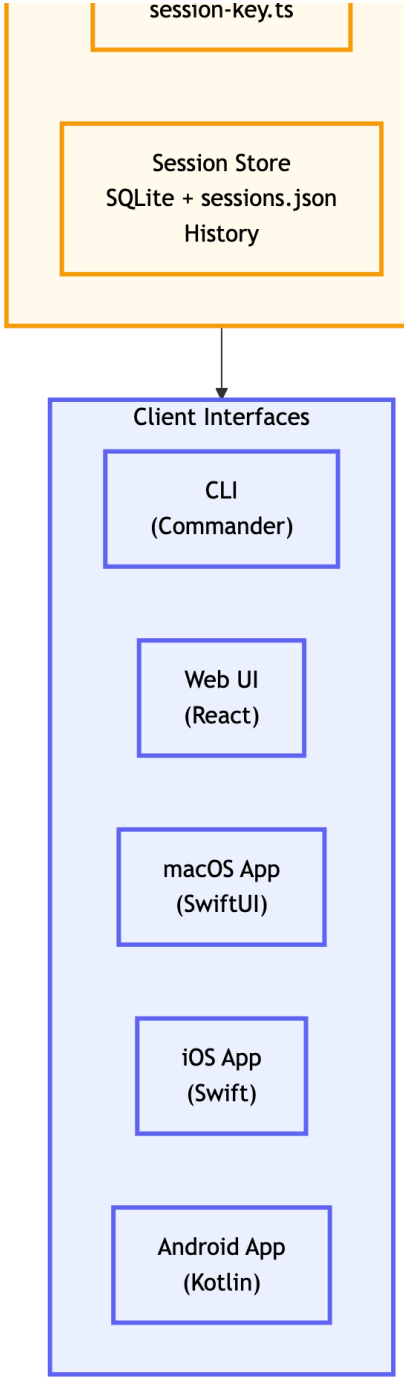
Table of Contents

System Architecture Diagram


Messaging Channels (Input
WhatsApp
Baileys
Telegram
grammY
Discord
discord.js
Slack
Bolt
Signal
signal-cli
iMessage
Native
+ More:
Google Chat, MS Teams,
Matrix, Zalo, WebChat
Support Layers
Config
Zod validation
JSON5/YAML
Credentials
Media
Audio, Image ops
PDF, Fetch, Store
Logging
tslog, Subsystem
Diagnostic events
Plugin System
Extensions (channels,
tools, memory)
Manifest registry
Runtime isolation


Gateway (Control Plane -
WebSocket Server
• TypeBox validation
• RPC protocol
• Device pairing
• Auth (token/password)
Chat Run Registry
• Session queue
• Agent event broadcast
• Concurrent lanes
• Rate limiting
Channel Manager
• All channel instances
• Health monitoring
• Reconnection logic
• Allowlists & policies
Subsystems
• Cron scheduler
• Browser manager
• Canvas server
• Discovery (mDNS)
• Tailscale
• Webhooks
• Maintenance


Agent Runtime (Pi
Execution Engine
pi-embedded-runner/run.ts
• Model invocation
• Tool streaming
• Failover logic
• Context window guards
Tool Pipeline
channel-tools.ts
• Browser, Canvas, Nodes
• Sessions, Skills, Search
• Messaging (send)
• Media processing
Model Auth & Providers
model-auth.ts
azure-openai-runtime.ts
• Anthropic, OpenAI
• Google, Azure
• OAuth & API keys
• Managed identity (Azure)
• Failover routing
Routing & Session
resolve-route.ts
bindings.ts


session-key.ts
Session Store
SQLite + sessions.json
History
Client Interfaces
CLI
(Commander)
Web UI
(React)
macOS App
(SwiftUI)
iOS App
(Swift)
Android App
(Kotlin)

Architecture Overview

Clawdbot is a multi-channel AI agent system built on a layered architecture. The system processes messages from various messaging platforms, routes them through a central gateway, executes AI agent logic, and delivers responses back to users through multiple client interfaces.

Layer 1: Messaging Channels (Input Layer)

Purpose: Accept messages from various messaging platforms

Supported Channels

- **WhatsApp** - Integration via Baileys library
- **Telegram** - Built with grammY framework
- **Discord** - Using discord.js
- **Slack** - Bolt framework integration
- **Signal** - Via signal-cli
- **iMessage** - Native macOS integration
- **Additional Platforms** - Google Chat, MS Teams, Matrix, Zalo, WebChat

Layer 2: Gateway (Control Plane)

Purpose: Central coordination and control plane

Endpoint: `ws://127.0.0.1:18789`

WebSocket Server

- TypeBox validation for message schemas
- RPC protocol implementation
- Device pairing management
- Authentication (token/password)

Chat Run Registry

- Session queue management
- Agent event broadcasting
- Concurrent lane handling
- Rate limiting enforcement

Channel Manager

- Manages all channel instances

- Health monitoring for channels
- Automatic reconnection logic
- Allowlist and policy enforcement

Subsystems

- Cron scheduler for automated tasks
- Browser automation manager
- Canvas rendering server
- mDNS-based service discovery
- Tailscale VPN integration
- Webhooks for external integrations
- Maintenance tasks (log pruning, cleanup)

Layer 3: Agent Runtime (Pi Embedded)

Purpose: Execute AI agent logic and tool calls

Execution Engine

`pi-embedded-runner/run.ts`

- Model invocation and response handling
- Tool call streaming
- Automatic failover logic
- Context window management and guards

Tool Pipeline

`channel-tools.ts`

- Browser automation tools
- Canvas rendering tools
- Node.js execution environment
- Session management tools
- Skills execution framework
- Web search integration
- Message sending capabilities

- Media processing (images, audio, video)

Model Auth & Providers

`model-auth.ts` , `azure-openai-runtime.ts`

- Multi-provider support: Anthropic, OpenAI, Google, Azure
- OAuth and API key resolution
- Azure Managed Identity support
- Intelligent failover routing between providers

Layer 4: Routing & Session Management

Purpose: Route messages and manage conversation state

Route Resolution

`resolve-route.ts`, `bindings.ts`, `session-key.ts`

- Determines which agent handles which conversation
- Manages channel-to-agent bindings
- Session key generation and mapping

Session Store

SQLite + `sessions.json`

- Persistent conversation history
- Session state management
- Message archival

Layer 5: Support Layers

Purpose: Cross-cutting infrastructure services

Configuration Management

- Zod schema validation
- JSON5/YAML configuration parsing
- Secure credential storage

Media Processing

- Audio transcription and synthesis
- Image manipulation and optimization
- PDF generation and parsing
- URL fetching and content extraction
- Media storage and retrieval

Logging System

- tslog-based structured logging
- Per-subsystem log levels

- Diagnostic event tracking

Plugin System

- Extension points for channels, tools, and memory
- Manifest-based plugin registry
- Runtime isolation for security
- Dynamic plugin loading

Layer 6: Client Interfaces

Purpose: User-facing interfaces to interact with the system

Available Interfaces

- **CLI** (Commander.js) - Command-line interface
- **Web UI** (React) - Browser-based interface
- **macOS App** (SwiftUI) - Native macOS application
- **iOS App** (Swift) - Native iPhone/iPad app
- **Android App** (Kotlin) - Native Android application

Data Flow

```
User Message ↓ [Messaging Channel] → Receives message via platform SDK ↓ [Gateway] → Validates, authenticates, routes to agent ↓ [Agent Runtime] → Executes AI logic, calls tools ↓ [Routing & Session] → Updates session state, stores history ↓ [Client Interface] → Displays response (if applicable)
```

Key Technologies

- **Language:** TypeScript, Swift, Kotlin
- **Runtime:** Node.js (Pi Embedded)
- **Communication:** WebSocket (RPC)
- **Storage:** SQLite, JSON files
- **AI Models:** Anthropic Claude, OpenAI GPT, Google Gemini, Azure OpenAI
- **Authentication:** OAuth 2.0, API keys, Azure Managed Identity
- **Validation:** TypeBox, Zod
- **Logging:** tslog

Security Features

- Token/password authentication
- Allowlist-based access control
- Runtime isolation for plugins
- Secure credential storage
- Rate limiting
- Message validation

Scalability Features

- Concurrent lane processing
- Session queue management
- Automatic failover routing
- Health monitoring and auto-reconnection
- Efficient context window management

Architecture Version: 1.0

Last Updated: 2026-02-07

Document Type: System Architecture Diagram